
Evaluating the World Model Implicit in a Generative Model

Keyon Vafa
Harvard University

Justin Y. Chen
MIT

Ashesh Rambachan
MIT

Jon Kleinberg
Cornell University

Sendhil Mullainathan
MIT

Abstract

Recent work suggests that large language models may implicitly learn world models. How should we assess this possibility? We formalize this question for the case where the underlying reality is governed by a deterministic finite automaton. This includes problems as diverse as simple logical reasoning, geographic navigation, game-playing, and chemistry. We propose new evaluation metrics for world model recovery inspired by the classic Myhill-Nerode theorem from language theory. We illustrate their utility in three domains: game playing, logic puzzles, and navigation. In all domains, the generative models we consider do well on existing diagnostics for assessing world models, but our evaluation metrics reveal their world models to be far less coherent than they appear. Such incoherence creates fragility: using a generative model to solve related but subtly different tasks can lead to failures. Building generative models that meaningfully capture the underlying logic of the domains they model would be immensely valuable; our results suggest new ways to assess how close a given model is to that goal.

1 Introduction

Large language models (LLMs) appear to have capacities that far exceed the next-token prediction task they were trained to perform [17, 39, 35]. Recent work suggests a reason: they are implicitly recovering high-fidelity representations of the underlying domains they are trained on [1, 20].

An algorithm that recovers a “world model” from sequence data would be extremely valuable. As an example, consider how one might build a navigation tool today: meticulously map each street and intersection, and then use a search algorithm to provide directions. The success of language models suggests an alternative approach: collect turn-by-turn sequences from trips in a city (e.g. “East North...”) and then train a sequence model on them. If the sequence model successfully recovers the world model, we would obtain a map of the city without ever mapping it and a routing algorithm simply by predicting the next turn. This example is not far-fetched: it is the reason language models are used in scientific domains such as protein generation, genetics and chemistry [7, 21, 3, 14, 6].

All of this relies on the presumption that the sequence model has recovered the true world model; but how can we test whether it actually has? Answering this question requires first defining what we mean by the true world model. Toshniwal et al. [36] and Li et al. [20] proposed a concrete and influential approach: study whether sequence models trained on board game transcripts (e.g. chess and Othello) recover the underlying game rules. Inspired by this approach, we consider the case where the underlying world can be summarized by a finite collection of states and rules governing transitions between the states; this includes many domains such as logic [19], location tracking [28, 9], games [36, 20], and several of the scientific applications described above. As a result, the “world” in these domains can be modeled as a deterministic finite automaton (DFA).

We show the difficulty in evaluating implicit world models. Consider an existing approach: for a given sequence, compare the next tokens outputted by the generative model to the set of valid next tokens for the state implied by that sequence [36, 20]. Though intuitive, this approach can fail to diagnose severe problems, and we illustrate this concretely. The classic Myhill-Nerode theorem [26, 27] provides intuition: every pair of distinct states can be distinguished by some sequence (admitted by one state but not the other). Unless those minimal distinguishing sequences are of length one, looking at the next *single* token outputted will not reliably assess whether the generative model has an accurate model of the underlying state.

The logic of Myhill-Nerode suggests two metrics for measuring whether a generative model effectively captures underlying states and transitions. The first metric summarizes *sequence compression*: under the DFA, sequences that lead to the same state must have the same continuations; so one can test whether the generative model has similar sequences of outputs when started on these two sequences. The second metric summarizes *sequence distinction*: under the DFA, two sequences that lead to distinct states should have distinct continuations; so one can test whether the generative model matches these distinct outputs when started at these two sequences. We formally define these metrics and provide model-agnostic procedures for calculating them when given query access to the true DFA.

To illustrate these ideas, we first take the stylized mapping example literally. We construct a turn-by-turn sequence dataset of taxi rides in New York City. We then assess to what extent transformers successfully recover the true street map of Manhattan. By the usual metrics, the transformers do very well: their predicted next-direction is a valid turn nearly 100% of the time and their state representations even appear to encode the current location of the ride. Our evaluation methods reveal they are very far from recovering the true street map of New York City. As a visualization, we use graph reconstruction techniques to recover each model’s implicit street map of New York City. The resulting map bears little resemblance to the actual streets of Manhattan, containing streets with impossible physical orientations and flyovers above other streets (see Figure 3). Because these transformers fail to recover the true street map of New York City, they are fragile for downstream tasks. While they sometimes have amazing route planning abilities, their performance breaks down when detours are introduced.

These results are not unique to maps and navigation. For both Othello and logic puzzles, we use our evaluation metrics to show language models can perform remarkably well on some tasks despite being far from recovering the true world model. These results demonstrate the importance of using theoretically-grounded evaluation metrics if our goal is to build language models that capture accurate world models of the domains they are trained in. We release our benchmark dataset of taxi rides in New York City along with software implementing our evaluation metrics.¹

Related work. Our paper builds on influential work studying whether generative models recover a world model in the context of games. Toshniwal et al. [36] and Li et al. [20] pioneered the study of games as a testbed for world model evaluation, studying tests for chess and Othello, respectively, which were further studied by Hazineh et al. [10] and Kuo et al. [18]. Our evaluation metrics apply to these games because they are DFAs. A common method for assessing whether a trained model has recovered a world model uses probes that assess whether a neural network’s representation can recover some real-world state [11, 19, 1, 16, 20]. By contrast, our evaluation metrics are model-agnostic: they’re based only on sequences. While the results from our evaluation metrics sometimes align with those used in existing work, they also reveal incoherence in world models that are not captured by existing diagnostics.

We study whether a language model trained on sequences of directions recovers the true underlying map. This question relates to other state tracking and navigation problems studied in the language modeling literature [31, 32]. For example, Patel & Pavlick [28] show that larger LLMs ground spatial concepts like cardinal directions to locations in a grid world and generalize to various grid layouts. Relatedly, Schumann & Riezler [30] demonstrate that transformer-based models can generate navigation instructions in language from underlying graphs. Additionally, Guan et al. [9] use LLMs to perform planning tasks from natural language descriptions. Our results suggest that LLMs can perform some of these tasks well (such as finding shortest paths between two points on a map) without having a coherent world model.

Additionally, our evaluation metrics compare the language accepted by a sequence model to that of an underlying DFA. Existing work studies whether transformers and other sequence models are theoretically capable of recognizing languages in different complexity classes [34, 4, 22, 23, 24].

¹<https://github.com/keyonvafa/world-model-evaluation>

Most relevant to our work, Liu et al. [22] show that low-depth transformers can theoretically represent any finite state automata, and show that transformers trained explicitly to predict their labeled states are capable of doing so. In contrast, our paper doesn't aim to study whether models are theoretically capable of recovering underlying automata or whether they can do so when given state labels. Instead, we provide metrics for assessing how closely a given model recovers the underlying DFA.

2 Framework

In this section, we lay out a framework to interface between generative sequence models and world models represented by deterministic finite automata. Both of these are built on the shared scaffolding of tokens, sequences (a.k.a. strings), and languages.

Tokens and sequences. We consider a finite alphabet Σ with tokens $a \in \Sigma$, and sequences $s = (a_1, a_2, \dots)$. Let Σ^* denote the collection of sequences on the alphabet.

Generative models. A *generative model* $m(\cdot): \Sigma^* \rightarrow \Delta(\Sigma)$ is a probability distribution over next-tokens given an input sequence. That is, $m(s) \in \Delta(\Sigma)$, and $m(a | s)$ is the probability assigned to token $a \in \Sigma$ given an input sequence s . Starting at a sequence s , the set of non-empty sequences the model can generate with positive probability is:

$$L^m(s) = \{a_1 a_2 \dots a_k : \forall j < k, m(a_{j+1} | s a_1 \dots a_j) > 0\}.$$

For simplicity, we write the equation above for next-tokens with nonzero probability, but in practice we set a minimum probability $\epsilon > 0$ corresponding to next-tokens with non-negligible probability.

Deterministic finite automata (DFA). We use standard notation for a deterministic finite state automaton $W = (Q, \Sigma, \delta, q_0, F)$ (see [Appendix C](#) for a complete definition). As a simplifying assumption, we consider the case where there is a special state q_{reject} with no outgoing transitions and $F = Q \setminus \{q_{\text{reject}}\}$ (i.e., the DFA accepts all valid states). An extended transition function $\hat{\delta}$ takes a state and a sequence, and it inductively applies δ to each token of the sequence. A token or a sequence is *valid* if and only if the output of δ or $\hat{\delta}$ respectively starting from q_0 is not q_{reject} .

We define $L^W(q)$ to be the set of valid, non-empty sequences that are accepted by the DFA starting at state q . We also define $q(s) \in F$ to be the state that sequence s leads to in the DFA starting from q_0 and $S(q) \subseteq \Sigma^*$ to be the collection of all sequences that lead from state q_0 to state q in the DFA.

2.1 Recovering world models

Throughout this paper we assume that the ground-truth sequences used to train and test a generative model belong to the language of a deterministic finite state automaton W . This generalizes past work (e.g., on assuming sequences come from legal moves in a game [36, 20]) and allows us to formally define world recovery.

Definition 2.1. A generative model $m(\cdot)$ **recovers the DFA** W if

$$\forall q \in F, \forall s \in S(q) : L^W(q) = L^m(s).$$

That is, recovery requires that a sequence can be generated with positive probability by the model $m(\cdot)$ if and only if the sequence is valid in the DFA W .

Recovery is defined at the language level. However, generative models are often built and evaluated token-by-token. It turns out that exact next-token prediction is enough for recovery of the language of the world model.

Definition 2.2. A generative model $m(\cdot)$ satisfies **exact next-token prediction** under the DFA W if

$$\forall q \in F, \forall s \in S(q), \forall a \in \Sigma : m(a | s) > 0 \iff \delta(q, a) \neq q_{\text{reject}}.$$

Proposition 2.3. A generative model $m(\cdot)$ recovers the DFA W if and only if it satisfies exact next-token prediction under the DFA W .

Proposition 2.3 (proof given in [Appendix A](#)) suggests a way to evaluate whether a generative model recovers the true DFA: assess the validity of next-token predictions. Existing world model diagnostics are motivated by this intuition; for example, one way that Toshniwal et al. [36] and Li et al. [20] assess world model recovery is by measuring the percent of top next-token predictions that are valid.

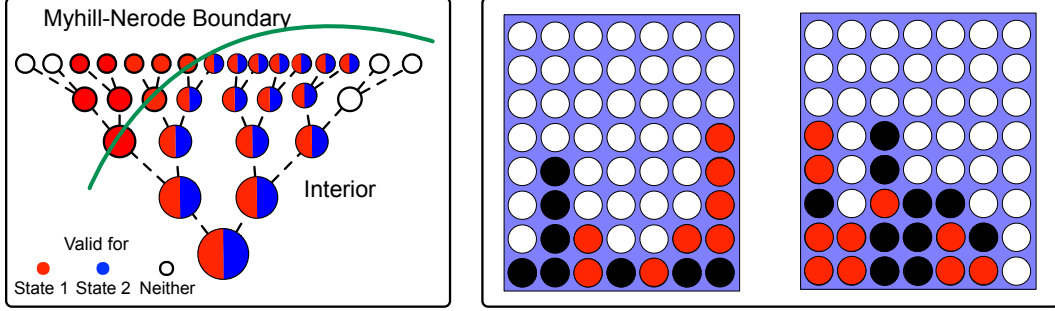


Figure 1: On the left, a visual depiction of a Myhill-Nerode boundary and interior. On the right, examples of two states for cumulative Connect-4. Both states have the same set of valid next moves. The shortest sequence in the Myhill-Nerode boundary has length 4, and the boundary contains sequences up to length 30. The interior contains approximately 8.8×10^{27} sequences of length 29 that do not distinguish the two boards.

2.2 Next-token prediction is a fragile metric for recovering structure

Next-token prediction, however, is a limited evaluation metric. While exact next-token prediction implies perfect world model recovery, being very nearly correct on next-token prediction does not mean having very nearly recovered the world model. This can be illustrated by a simple example.

Example: Cumulative Connect-4. Consider a vertical grid with n rows and 7 columns. Two players take turns dropping a disk in a column, and they can choose any column that contains less than n disks. When a disk is dropped in a column, it occupies the bottom-most position that isn't occupied by another disk, and it remains in that position for the full game. The game continues until the entire board is filled, for $7n$ moves, regardless of whether a player has achieved four in a row. Games are represented as sequences of moves, where each sequence has $7n$ tokens and each token is an integer between 1 and 7 indicating the column the disk is placed in. Here, $\Sigma = \{1, \dots, 7\}$ denotes the columns and the state corresponds to the count in each column. A column is a valid move if that column is not already filled.

Consider a generative model that outputs $\{1, \dots, 7\}$ with uniform probability given any sequence, i.e. $m(a | s) = m(a' | s') = 1/7$ for all $a, a' \in \Sigma$ and $s, s' \in \Sigma^*$. This model clearly encodes no information about the board. However, for any board where there are no columns filled, this model provides a valid next move (e.g., the right panel of Figure 1), and so it will be a near-perfect next-token predictor when n is large. For example, when $n = 1000$, it predicts a valid next move for more than 99% of all states. Metrics based on next-token prediction will imply this algorithm is close to recovering a world model.

2.3 The Myhill-Nerode interior and boundary

Cumulative Connect-4 points to a general fragility in next-token prediction as an evaluation metric that can be understood in the context of the Myhill-Nerode theorem [26, 27], a classic result from language theory. The Myhill-Nerode theorem states that the sets of sequences accepted by a minimal DFA starting at two distinct states are distinct (see Appendix C for a full statement). More formally, for states $q_1 \neq q_2$, we have $L^W(q_1) \neq L^W(q_2)$. However, while distinct, the two sets may exhibit a great deal of overlap. Cumulative Connect-4 exhibits this behavior; any board for which there are less than k disks in each column will have the same set of valid moves for the next $n - k$ moves. This intuition motivates a pair of definitions:

Definition 2.4. Given a DFA W , the **Myhill-Nerode interior** for the pair $q_1, q_2 \in F$ is the set of sequences accepted when starting at both states:

$$\text{MNI}^W(q_1, q_2) = \{s \in \Sigma^* \mid s \in L^W(q_1) \cap L^W(q_2)\}.$$

The **Myhill-Nerode boundary** is the set of minimal suffixes accepted by a DFA at q_1 but not q_2 :

$$\text{MNB}^W(q_1, q_2) = \{s = a_1 a_2 \dots a_k \mid s \in L^W(q_1) \setminus L^W(q_2) \text{ and } \forall j < k : a_1 \dots a_j \in \text{MNI}^W(q_1, q_2)\}.$$

Figure 1 depicts an example Myhill-Nerode interior and boundary for cumulative Connect 4. Sequences on the interior are accepted by both states; it is only when we reach the boundary that these states will be distinguishable. Thus, models that pool together states with large interiors will

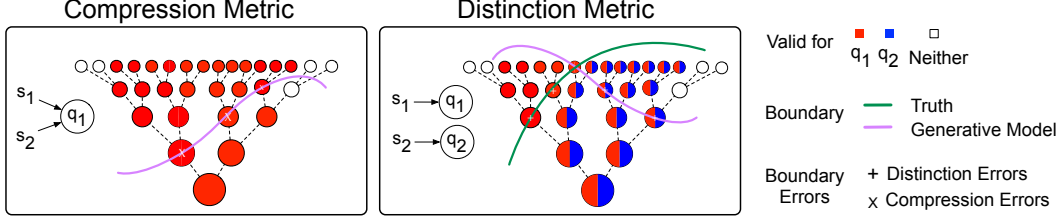


Figure 2: A visual depiction of our two evaluation metrics. A compression error is a model failing to recognize that two sequences that result in the same state should accept the same suffixes. A distinction error is a model failing to find the right distinguishing suffixes for two sequences that lead to different states. Our metrics measure errors at the boundary, which are visually depicted above.

perform well on next-token prediction tests; this is why the simple generative model succeeds in the cumulative Connect-4 example. To properly differentiate states, we must consider sequences that are long enough to be differentiated. In the remainder of the paper, we (i) use the Myhill-Nerode logic to develop new evaluation metrics and (ii) apply these to several applications.

2.4 Compression and distinction metrics for evaluating world models

We propose metrics to evaluate a model’s implicit world model by comparing the true Myhill-Nerode boundary to the one implied by the model.

Definition 2.5. For two sequences s_1, s_2 , the **Myhill-Nerode boundary implied by model $m(\cdot)$** is

$$\text{MNB}^m(s_1, s_2) = \{x = x_1 \dots x_k \mid x \in L^m(s_1) \setminus L^m(s_2) \text{ and } \forall j < k : x_1 \dots x_j \in L^m(s_1) \cap L^m(s_2)\}. \quad (1)$$

This is the set of minimal suffixes that are accepted by the model conditioned on s_1 but not s_2 . Since we now focus on the generative model rather than the DFA, the definition refers to pairs of sequences rather than to pairs of states.

Our evaluation metrics summarize how well a generative model identifies sequences that distinguish a given pair of states. Given a pair of states q_1 and q_2 , the metric is formed by first sampling sequences that lead to each state, $s_1 \in S(q_1)$ and $s_2 \in S(q_2)$. We then calculate the true Myhill-Nerode boundary between the states and the model’s boundary between the sequences. Our metrics then compare the resulting boundaries using two statistics as building blocks:

Definition 2.6. The **boundary recall** of generative model $m(\cdot)$ with respect to a DFA W is defined as

$$\frac{|\text{MNB}^W(q_1, q_2) \cap (L^m(s_1) \setminus L^m(s_2))|}{|\text{MNB}^W(q_1, q_2)|}, \quad (2)$$

and the **boundary precision** is defined as

$$\frac{|\text{MNB}^m(s_1, s_2) \cap (L^W(q_1) \setminus L^W(q_2))|}{|\text{MNB}^m(s_1, s_2)|}. \quad (3)$$

Notice that boundary recall and boundary precision are not affected by whether the Myhill-Nerode interior is large between the two states. Returning to cumulative Connect-4, the simple generative model that outputs $\{1, \dots, 7\}$ with equal probability will perform poorly on these metrics; its recall will be 0 for all pairs of distinct states.

Based on the building blocks of recall and precision, we construct evaluation metrics to summarize whether the generative model correctly *compresses* sequences that arrive at the same state under the DFA and correctly *distinguishes* sequences that arrive at different states under the DFA. These two metrics correspond to different methods of sampling state pairs.

Sequence compression metric. To evaluate sequence compression, we sample equal state pairs $q_1 = q_2$. Since a DFA provides multiple ways to arrive at the same state, this test assesses whether a generative model recognizes that two sequences correspond to the same state. For example, in cumulative Connect-4, there may be multiple sequences that arrive at the same board position. Recall is undefined for equal states because there is no true boundary, so our compression metric only reports

precision, averaged over states sampled uniformly at random (we say a generative model’s precision is 1 if its boundary is correctly empty).

Sequence distinction metric. To evaluate sequence distinction, we sample distinct state pairs, i.e. $q_1 \neq q_2$. Here, there must be a true boundary, so we test how well a generative model recovers it. We report both precision and recall averaged over state pairs sampled uniformly at random.

Both metrics are depicted in Figure 2. Although we have defined a generative model as accepting all sequences it assigns positive probability to, in practice sequence models are regularized to assign all sequences nonzero probability. Our evaluation metrics therefore depend on an acceptance threshold parameter $\epsilon > 0$. In practice, we explore sensitivity to different values of ϵ and other acceptance mechanisms. We present ablations and other details in more depth in Section 3 and Appendix E.

3 Illustration: Do Transformers Recover the Street Map of New York City?

To illustrate these metrics, we create a dataset consisting of taxi rides in New York City. We process each ride into sequences of turn-by-turn directions and train transformers to predict the next direction. We show that transformers trained on these sequences have surprising route planning abilities: they not only find valid routes between two intersections but usually find the shortest path.

We then examine the underlying world model of the trained models. Despite the route planning capabilities of these models, our metrics reveal that their underlying world models are incoherent. Using a graph reconstruction technique, we show that each model’s implicit street map of New York City bears little resemblance to the actual map. Finally, we demonstrate that the route planning capabilities of these models break down when detours are introduced, a consequence of their incoherent world models.

3.1 Data and models

We base our analysis on a dataset of taxi rides released by the NYC Taxi & Limousine Commission, containing the latitude and longitude of each ride’s pickup and dropoff location in Manhattan. Each taxi ride obeys a true world model: the weighted graph corresponding to the system of intersections and streets in New York City. The graph is defined as $G = (V, E, W)$, where V is the set of intersections, E the set of streets, and $W : E \rightarrow \mathbb{R}^+$ a weighting function containing the distance of each street.² Each edge is labeled corresponding to its cardinal direction, represented as a function $D : V \times V \rightarrow \{\square, N, S, E, W, NE, NW, SE, SW\}$ with $\square$ indicating that the edge does not exist. Each intersection has at most one edge in each direction. The graph has 4580 nodes (i.e. intersections) and 9846 edges (i.e. streets).

A traversal is a sequence of nodes where an edge exists between each consecutive node in the sequence. To study how the construction of traversals affects the resulting generative model, we consider three different approaches. *Shortest paths* constructs traversals by finding the shortest path between two nodes. Since these may not be reflective of real-world traversals due to traffic conditions, *noisy shortest paths* constructs multiple shortest paths by perturbing the magnitude of each edge weight in the underlying graph. Finally, *random walks* samples random traversals instead of approximating shortest paths. See Appendix F for details.

We convert each traversal into a sequence of directions. Each sequence begins with the origin and destination, followed by the cardinal directions in the traversal, and concludes with a special end-of-sequence token. Figure 5 gives an example of a set directions and the corresponding path. Since this language corresponds to a DFA W with $|V|^2 + 1$ accept states, corresponding to all combinations of current intersection/destination intersection pairs and an additional end state, we can apply the evaluation metrics in Section 2.4.

We randomly split data into train and test splits, ensuring no origin-destination pair is in both train and test sets. We include all sequences containing less than 100 directions. Our training sets consist of 2.9M sequences (120M tokens) for shortest paths; 31M sequences (1.7B tokens) for noisy shortest paths; and 91M sequences (4.7B tokens) for random walks. We train two types of transformers [38] from scratch using next-token prediction for each dataset: an 89.3M parameter model consisting of 12 layers, 768 hidden dimensions, and 12 heads; and a 1.5B parameter model consisting of 48 layers, 1600 hidden dimensions, and 25 heads. We follow the architecture of GPT-2 for each model [29]. We train models on

²A real-world intersection may be represented as multiple intersections here. For example, if a turn is only valid from one direction, it is represented as two different nodes.

	Existing metrics		Proposed metrics		
	Next-token test	Current state probe	Compression precision	Distinction precision	Distinction recall
Untrained transformer	0.03 (0.00)	0.10 (0.00)	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Shortest paths	1.00 (0.00)	0.91 (0.00)	0.10 (0.01)	0.35 (0.02)	0.20 (0.01)
Noisy shortest paths	1.00 (0.00)	0.92 (0.00)	0.05 (0.01)	0.37 (0.02)	0.24 (0.01)
Random walks	1.00 (0.00)	0.99 (0.00)	0.50 (0.02)	0.99 (0.00)	1.00 (0.00)
True world model	1.00	—	1.00	1.00	1.00

Table 1: Sequence compression and distinction metrics for world models compared to existing metrics (standard errors in parentheses). Models that do well on existing metrics can perform poorly on ours.

8 A100 GPUs. For each dataset, we analyze the model with the best held-out performance: the 89.3M parameter model for shortest paths, and the 1.5B parameter for noisy shortest paths and random walks.

3.2 Evaluating world models

To assess their capabilities, we first assess whether the trained models can recover the shortest paths between unseen (origin, destination) pairs. We prompt each model with (origin, destination) pairs from the test set and use greedy decoding to generate a set of directions. All models consistently generate valid traversals — between 96% and 99%. Impressively, 97% of the sequences generated by the shortest paths model are the true shortest path, and 94% of the sequences generated by the model trained on noisy shortest paths find a shortest path for one of the noisy graphs used to generate data. Figure 5 provides an example of a shortest path traversal.

To assess whether these capabilities correspond to coherent implicit world models, we first consider two existing diagnostics [36, 20]. The **next-token test** assesses whether a model, when conditioned on each subsequence in the test set, predicts a legal turn for its top-1 predicted next-token. In our example, a directional move is legal if a street in the direction exists at the current intersection. Predicting the end token is only legal if the traversal implied by the sequence is at the listed destination. Meanwhile, the **current-state probe** trains a probe [11] from a transformer’s representation to predict the current intersection implied by the directions so far. We train a linear probe on a transformer’s last layer representation.

To implement the sequence compression metric, we randomly sample states (i.e., [intersection, destination] pairs) and two distinct traversals (i.e. prefixes) that arrive at each state. We then assess whether a model correctly admits the same suffixes for each prefix. We average over pairs of prefixes to report a score for each state and average over states to report a final score. To implement the sequence distinction metrics, we sample pairs of distinct states and traversals (i.e. prefixes) that arrive at each state, comparing the model’s approximate Myhill-Nerode boundary to the true one. We average over pairs of prefixes to report a score for each pair of states, and average over 1000 randomly sampled state pairs to report a final scores. Both metrics depend on a threshold parameter ϵ : a prefix is only sampled or accepted if the model’s assigned probability for each token is above ϵ . Here, we consider $\epsilon = 0.01$ for all models and metrics. We describe implementation details, provide parameter ablations, and consider other acceptance rules (e.g. top-p and top-k) in Appendix E.

Table 1 summarizes our results. As references, we compare each trained transformer to a randomly initialized transformer baseline following Li et al. [20] as well as to the true world model. The three trained transformers perform exceptionally well on existing diagnostics; nearly 100% of next-token predictions are valid and the probe recovers the true intersection for more than 90% of examples.³

Our evaluation metrics, however, reveal that these existing diagnostics are incomplete. All trained transformers perform poorly on sequence compression, frequently failing to recognize that two prefixes leading to the same state should admit the same continuations. Even the transformer trained on random walks, which sees many distinct types of traversals during training, fails to compress prefixes for half the states. For the sequence distinction metrics, the transformers trained on shortest paths or noisy shortest paths perform poorly. In contrast, the transformer trained on random walks performs well on the sequence distinction metric. Both metrics are therefore valuable for evaluating world models; a model can perform well on one metric and poorly on the other. Here, a model that

³While the next-token test accuracy is rounded to 100%, no model performs perfectly.

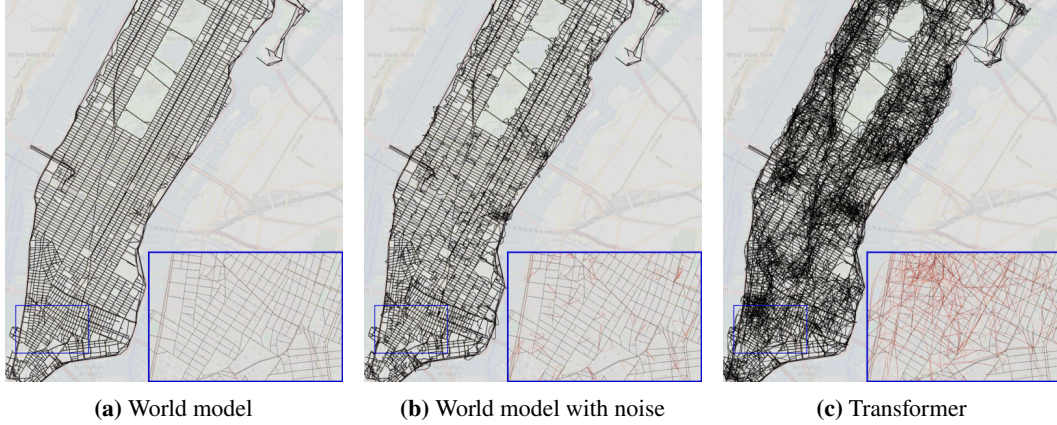


Figure 3: Reconstructed maps of Manhattan from sequences produced by three models: the true world model (left), the true world model corrupted with noise (middle), and a transformer trained on random walks (right). Edges exit nodes in their specified cardinal direction. In the zoomed-in images, edges belonging to the true graph are black and false edges added by the reconstruction algorithm are red. We host interactive reconstructed maps from transformers at the following links: [shortest paths](#), [noisy shortest paths](#), and [random walks](#).

distinguishes separate states at a high rate fails to recognize that two prefixes that lead to the same state should have the same valid continuations.

3.3 Reconstructing implicit maps

Our evaluation metrics point to deficiencies in recovering world models. We now show that these metrics reveal underlying incoherence. In the maps setting, the state structure of the true world model is easy to interpret and visualize: it is defined by the map itself. We attempt to “reconstruct” the map implied by sequences sampled from each generative model.

Reconstruction is an open-ended problem: the generative model produces directions between an origin and destination that do not necessarily correspond to a fixed graph over the intersections in Manhattan. To narrow the scope, our goal is to produce a visually interpretable reconstructed map. To that end, we fix the reconstructed graph to have the same set of vertices as the true world model, corresponding to intersections in Manhattan, and ensure that the reconstruction algorithm returns a map consistent with the true model whenever it is run on valid sequences. Further, (a) we enforce each node has at most one outgoing edge of any direction, (b) we limit the maximum degree of each node, and (c) we limit the Euclidean distance spanned by any edge. Altogether, our reconstruction algorithm gives the generative model the benefit of the doubt, attempting to reconstruct edges belonging to the true map until forced to do otherwise in order to map a generated sequence. The algorithm is detailed in [Appendix B](#).

[Figure 3](#) shows three reconstructed maps using sequences generated by the transformer trained on random walks. The sequences underlying each map are generated by randomly sampling 6400 (origin, destination) pairs and then sampling the model’s traversal for each pair ([Appendix G](#) shows similar results for when the distribution of origin/destination pairs follows the sampling distribution used to train each model). On the left is the reconstructed map on only sequences which are valid under the true world model. On the right is the reconstructed map using the transformer’s sequences. The transformer’s underlying world model is incoherent; it recovers streets whose orientations are physically impossible (e.g. labeled NW but facing east) and require flyovers above other streets.

To show that this map is not the product of a model that has the right world model but makes a few transcription errors, we artificially corrupt sequences drawn from the true model. With probability equal to the probability of an error for the random walks transformer, we randomly re-label an edge in a sequence consistent with the world model. The middle panel of [Figure 3](#) shows the reconstructed graph. It is much closer to the true world model than the transformer (which makes errors at the same rate). While these results are for random walks and one setting of graph reconstruction, [Appendix G](#) shows maps for the other models and different reconstruction settings. All settings recover incoherent underlying maps.

		Probability of detour				
		0%	1%	10%	50%	75%
Random detours	Shortest paths	0.99 (0.01)	0.69 (0.05)	0.08 (0.03)	0.00 (0.00)	0.00 (0.00)
	Noisy shortest paths	0.96 (0.02)	0.52 (0.05)	0.03 (0.02)	0.00 (0.00)	0.00 (0.00)
	Random walks	0.99 (0.01)	0.99 (0.01)	1.00 (0.00)	0.97 (0.02)	0.74 (0.04)
	True world model	1.00	1.00	1.00	1.00	1.00
Adversarial detours	Shortest paths	0.99 (0.01)	0.66 (0.05)	0.06 (0.02)	0.00 (0.00)	0.00 (0.00)
	Noisy shortest paths	0.96 (0.02)	0.64 (0.05)	0.04 (0.02)	0.00 (0.00)	0.00 (0.00)
	Random walks	0.99 (0.01)	1.00 (0.00)	1.00 (0.00)	0.93 (0.03)	0.51 (0.05)
	True world model	1.00	1.00	1.00	1.00	1.00

Table 2: The fraction of traversals that are valid when detours are introduced (standard errors in parentheses).

3.4 Implication of failing to recover the world model: detour fragility

Does it matter that the transformer has an incoherent world model? After all, it does very well at the practical task of finding shortest paths. Here we look at a slightly adjacent task and consider a driver facing detours while driving; how well does the model re-route?

Concretely, we feed each transformer an (origin, destination) pair from the test set and greedily decode a traversal. But with probability p for each token, we add one of two kinds of detours: for “random detours”, the model’s proposed token is replaced with a randomly chosen (true) valid token; for “adversarial detours”, it is replaced with the model’s lowest ranked valid token. We always ensure a valid path to the destination exists (shorter than length 100) after each detour. [Table 2](#) shows the fraction of valid traversals produced. While all models perform well initially, detours erode performance, illustrating how faulty world models can prove problematic. Notably, the transformer trained on random walks is more robust to detours than models trained on (noisy) shortest paths, mirroring its advantage on our proposed evaluation metrics. The similarity between model performance on these evaluation metrics and detour robustness illustrates the effectiveness of our proposed metrics for assessing world model recovery.

4 Other Applications: Othello and Logic Puzzles

We apply our evaluation metrics to two other settings: sequence models trained on games of Othello and large language models prompted to solve logic puzzles. In both cases, our framework finds the same type of incoherence that we found in the previous section for maps.

Othello. Li et al. [20] study the question of evaluating world models in the context of Othello, a board game that consists of players placing tokens on an 8x8 board. They train transformers on game transcripts to predict the next move of each game. They show these models perform well on both the next-token test and current-state probe considered in [Section 2.4](#). Since the true Othello game can be represented as a DFA, we can apply our world model evaluation metrics. The sequence compression metric assesses whether openings that lead to the same board position have the same predicted next moves, while the sequence distinction metrics assess whether the model can differentiate two distinct boards.

We apply our metrics to the two Othello sequence models considered by Li et al. [20]: one trained on real games from Othello championship tournaments and another trained on synthetic games. [Table 3](#) in [Appendix D](#) reports the metrics in both settings. The model trained on real games performs poorly on both compression and distinction metrics, failing to group together most pairs of game openings that lead to the same board. In contrast, the model trained on synthetic games performs well on both metrics. This discernment is not captured by the existing metrics, which show both models performing similarly. We validate this discernment by performing a “detours” exercise for Othello in [Table 4](#) in [Appendix D](#); while the model trained on synthetic data produces near-perfect games regardless of detours, the model trained on championship data fails immediately. Similar to the navigation setting, we again find that models trained on random/synthetic data recover more structure than those trained on real-world data.

Logic puzzles. We consider an additional application involving LLMs. Our metrics require that the ground truth language can be expressed as a DFA, so we consider a “seating arrangement” logic puzzle similar to those in Suzgun et al. [35]. There are n seats and n individuals. The vocabulary consists of statements like “Person ‘A’ is sitting in seat 1” and “Person ‘B’ is two seats away from

	Capabilities	Proposed metrics	
	Task accuracy	Compression precision	Distinction recall
Example task prompt There are 3 individuals named A, B, and C, and there are 3 seats, positioned 1-3. We have the following statements: 1. B is in seat 3 2. B is 1 seat away from A Based on this information, where is C seated? You can use chain-of-thought reasoning.	Llama-2 (70B)	0.77 (0.03)	0.08 (0.03)
	Llama-3 (8B)	0.85 (0.02)	0.18 (0.04)
	Llama-3 (70B)	0.98 (0.00)	0.25 (0.04)
	Mixtral-8x22B	0.88 (0.01)	0.35 (0.05)
	Qwen 1.5 (72B)	0.88 (0.02)	0.21 (0.04)
	Qwen 1.5 (110B)	0.98 (0.00)	0.53 (0.05)
	GPT-3.5 (turbo)	0.83 (0.02)	0.33 (0.05)
	GPT-4	1.00 (0.00)	0.21 (0.04)
	True world model	1.00	1.00

Figure 4: On the left, an example given to large language models to assess task capabilities. On the right, each model’s average task performance along with their results on our proposed metrics. Models are very capable of solving logic puzzles despite not having a coherent world model.

Person ‘C’. A state is the set of seating arrangements that are consistent with all of the statements so far, and a statement is valid if it doesn’t contradict all arrangements in the given state.

We analyze Llama 2 (70B) [37], Llama-3 (8B and 70B), Mixtral (8x22B Instruct) [15], Qwen 1.5 Chat (72B and 110B) [2], GPT-3.5 turbo, and GPT-4. We consider $n=3$ individuals. We first assess whether the LLMs solve the logic puzzle task when the seating arrangement is fully specified by the statements. Figure 4 shows that most LLMs perform well at this task; GPT-4 is accurate on all examples. We then apply our metrics, assessing if each LLM compresses correctly (whether two sets of statements that lead to the same state lead to the same assessments) and has high recall for distinction (we do not compute precision for distinction because it is too expensive to approximate each LLM’s Myhill-Nerode boundary). See Appendix E for further discussion.

Figure 4 shows the results averaged over 100 samples. While most LLMs can solve the logic puzzle when it’s fully specified, they perform poorly on the compression and distinction metrics: no model has a compression precision higher than 40%. More than half the time a model is conditioned on two sequences with the same set of viable states, it asserts that different continuations are allowed for each sequence; see Figure 8 for an example. No model has distinction recall higher than 0.60. These results bring up an interesting point: LLMs can perform well at some logic tasks (such as when the seating arrangement is fully specified) without having a coherent world model.

5 Conclusion

In order to build high-fidelity algorithms that meaningfully capture the logic of the problems they model, we need ways to measure how close we are to that goal. This paper suggests theoretically grounded metrics for assessing the world models implicit inside generative models. Applications to maps, games, and logic puzzles suggest these metrics are both feasible to implement and insightful. Our results show that generative models can perform impressive tasks with incoherent world models (e.g. provide directions for taxi rides). But this incoherence makes them fragile for other tasks (e.g. providing directions when there are detours). This incoherence is also problematic when we hope to use a generative model to learn something latent about the world in scientific domains.

Our primary limitation is the focus on DFAs. While it is suitable for many applications like games, logic, and state tracking, extending it would be quite valuable, e.g. to situations where the underlying world model is more complicated than a DFA or is unknown. We suspect that the core ideas related to sequence compression and sequence distinction generalize to these richer settings, but leave that to future work.

Acknowledgements

Keyon Vafa is supported by the Harvard Data Science Initiative. Justin Chen is supported by an NSF Graduate Research Fellowship under Grant No. 174530. Jon Kleinberg is supported in part by a Vannevar Bush Faculty Fellowship, a Simons Collaboration grant, and a grant from the MacArthur Foundation. We thank the Chicago Booth School of Business for generous support. We thank

Foundry⁴ for providing the compute required to conduct this research. We also thank Sarah Bentley, Juan Carlos Perdomo, and Neekon Vafa for helpful comments and feedback.

References

- [1] Abdou, M., Kulmizev, A., Hershcovich, D., Frank, S., Pavlick, E., and Søgaard, A. Can language models encode perceptual structure without grounding? A case study in color. *arXiv preprint arXiv:2109.06129*, 2021.
- [2] Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X., Fan, Y., Ge, W., Han, Y., Huang, F., et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- [3] Benegas, G., Batra, S. S., and Song, Y. S. DNA language models are powerful predictors of genome-wide variant effects. *Proceedings of the National Academy of Sciences*, 120(44): e2311219120, 2023.
- [4] Bhattamishra, S., Ahuja, K., and Goyal, N. On the ability and limitations of transformers to recognize formal languages. *arXiv preprint arXiv:2009.11264*, 2020.
- [5] Boeing, G. Modeling and analyzing urban networks and amenities with OSMnx. 2024.
- [6] Boiko, D. A., MacKnight, R., Kline, B., and Gomes, G. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023.
- [7] Chowdhury, R., Bouatta, N., Biswas, S., Floristean, C., Kharkar, A., Roy, K., Rochereau, C., Ahdritz, G., Zhang, J., Church, G. M., Sorger, P. K., and AlQuraishi, M. Single-sequence protein structure prediction using a language model and deep learning. *Nature Biotechnology*, 40(11): 1617–1623, 2022.
- [8] Fan, A., Lewis, M., and Dauphin, Y. Hierarchical neural story generation. *arXiv preprint arXiv:1805.04833*, 2018.
- [9] Guan, L., Valmeekam, K., Sreedharan, S., and Kambhampati, S. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. *Advances in Neural Information Processing Systems*, 36:79081–79094, 2023.
- [10] Hazineh, D. S., Zhang, Z., and Chiu, J. Linear latent world models in simple transformers: A case study on Othello-GPT. *arXiv preprint arXiv:2310.07582*, 2023.
- [11] Hewitt, J. and Liang, P. Designing and interpreting probes with control tasks. *arXiv preprint arXiv:1909.03368*, 2019.
- [12] Hewitt, J., Manning, C. D., and Liang, P. Truncation sampling as language model desmoothing. *arXiv preprint arXiv:2210.15191*, 2022.
- [13] Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.
- [14] Jablonka, K. M., Schwaller, P., Ortega-Guerrero, A., and Smit, B. Leveraging large language models for predictive chemistry. *Nature Machine Intelligence*, pp. 1–9, 2024.
- [15] Jiang, A. Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D. S., Casas, D. d. l., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [16] Jin, C. and Rinard, M. Evidence of meaning in language models trained on programs. *arXiv preprint arXiv:2305.11169*, 2023.
- [17] Kıcıman, E., Ness, R., Sharma, A., and Tan, C. Causal reasoning and large language models: Opening a new frontier for causality. *arXiv preprint arXiv:2305.00050*, 2023.
- [18] Kuo, M.-T., Hsueh, C.-C., and Tsai, R. T.-H. Large language models on the chessboard: A study on ChatGPT’s formal language comprehension and complex reasoning skills. *arXiv preprint arXiv:2308.15118*, 2023.
- [19] Li, B. Z., Nye, M., and Andreas, J. Implicit representations of meaning in neural language models. *arXiv preprint arXiv:2106.00737*, 2021.

⁴<https://www.mlfoundry.com/>

- [20] Li, K., Hopkins, A. K., Bau, D., Viégas, F., Pfister, H., and Wattenberg, M. Emergent world representations: Exploring a sequence model trained on a synthetic task. In *International Conference on Learning Representations*, 2023.
- [21] Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., Smetanin, N., Verkuil, R., Kabeli, O., Shmueli, Y., dos Santos Costa, A., Fazel-Zarandi, M., Sercu, T., Candido, S., and Rives, A. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.
- [22] Liu, B., Ash, J. T., Goel, S., Krishnamurthy, A., and Zhang, C. Transformers learn shortcuts to automata. *arXiv preprint arXiv:2210.10749*, 2022.
- [23] Merrill, W. and Sabharwal, A. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545, 2023.
- [24] Merrill, W., Petty, J., and Sabharwal, A. The illusion of state in state-space models. *arXiv preprint arXiv:2404.08819*, 2024.
- [25] Murray, K. W. 2014 New York City taxi trips. <https://www.kaggle.com/datasets/kentonnlp/2014-new-york-city-taxi-trips>, 2017. Accessed: 2024-10-24.
- [26] Myhill, J. Finite automata and the representation of events. *WADD Technical Report*, 57: 112–137, 1957.
- [27] Nerode, A. Linear automaton transformations. *Proceedings of the American Mathematical Society*, 9(4):541–544, 1958.
- [28] Patel, R. and Pavlick, E. Mapping language models to grounded conceptual spaces. In *International Conference on Learning Representations*, 2021.
- [29] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.
- [30] Schumann, R. and Riezler, S. Generating landmark navigation instructions from maps as a graph-to-text problem. *Association for Computational Linguistics*, 2021.
- [31] Schumann, R. and Riezler, S. Analyzing generalization of vision and language navigation to unseen outdoor areas. *Association for Computational Linguistics*, 2022.
- [32] Schumann, R., Zhu, W., Feng, W., Fu, T.-J., Riezler, S., and Wang, W. Y. VELMA: Verbalization embodiment of LLM agents for vision and language navigation in street view. In *AAAI Conference on Artificial Intelligence*, 2024.
- [33] Sipser, M. *Introduction to the Theory of Computation, Third Edition*. Cengage Learning, 2013.
- [34] Suzgun, M., Belinkov, Y., and Shieber, S. M. On evaluating the generalization of LSTM models in formal languages. *arXiv preprint arXiv:1811.01001*, 2018.
- [35] Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q. V., Chi, E. H., Zhou, D., et al. Challenging BIG-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.
- [36] Toshniwal, S., Wiseman, S., Livescu, K., and Gimpel, K. Chess as a testbed for language model state tracking. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pp. 11385–11393, 2022.
- [37] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [38] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Neural Information Processing Systems*, 2017.
- [39] Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E. H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J., and Fedus, W. Emergent abilities of large language models. *arXiv preprint arXiv:2206.07682*, 2022.
- [40] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. Chain-of-thought prompting elicits reasoning in large language models. *Neural Information Processing Systems*, 35:24824–24837, 2022.